# Simple Checkers Java Programming Guide

Use this as a checklist while programming each file. It only lists what each class is responsible for, the main data it needs, and the methods you will likely need.

## Piece.java

**What it is:** Represents one checker piece.

**Fields / data it needs:**

- color - which player the piece belongs to
- king - whether the piece has been promoted to a king

**Constructor:**

- Piece(color, king) or Piece(color) with king starting as false

**Main methods you will likely need:**

- getColor()
- isKing()
- setKing(boolean)

**Matching test file:** PieceTest.java

## Move.java

**What it is:** Represents one move from a starting square to an ending square.

**Fields / data it needs:**

- startRow
- startCol
- endRow
- endCol

**Constructor:**

- Move(startRow, startCol, endRow, endCol)

**Main methods you will likely need:**

- getStartRow()
- getStartCol()
- getEndRow()
- getEndCol()

**Matching test file:** MoveTest.java

## CheckersBoard.java

**What it is:** Stores the 8 x 8 board and keeps track of where every Piece is located.

**Fields / data it needs:**

- Piece[][] board = an 8 x 8 two-dimensional array

**Constructor:**

- CheckersBoard() - creates the board and places the starting pieces

**Main methods you will likely need:**

- getPiece(row, col)
- placePiece(row, col, piece)
- movePiece(move)
- removePiece(row, col)
- isInsideBoard(row, col)
- resetBoard()

**Matching test file:** CheckersBoardTest.java

## MoveValidator.java

**What it is:** Decides whether a requested move follows the rules of checkers.

**Fields / data it needs:**

- Usually does not need much stored data; it works with a Move, Piece, and CheckersBoard

**Constructor:**

- Can use a default constructor, or receive the board

**Main methods you will likely need:**

- isValidMove(move, board, currentPlayer)
- isSimpleMove(...)
- isCaptureMove(...)
- isCorrectDirection(...)
- canKingMove(...)
- hasCaptureAvailable(...)

**Matching test file:** MoveValidatorTest.java

## CheckersGame.java

**What it is:** Controls the overall game. This is where turns, valid moves, captures, kings, and winning come together.

**Fields / data it needs:**

- CheckersBoard board
- MoveValidator validator
- currentPlayer
- gameOver / winner

**Constructor:**

- CheckersGame() - creates a new board, validator, and starting turn

**Main methods you will likely need:**

- makeMove(move)
- switchTurn()
- getCurrentPlayer()
- checkForKing(piece, row)
- checkWin()
- isGameOver()
- getBoard()

**Matching test file:** CheckersGameTest.java

## CheckersView.java

**What it is:** JavaFX display only. It shows the board and lets the user select squares. The actual rules stay in CheckersGame and MoveValidator.

**Fields / data it needs:**

- Reference to CheckersGame
- JavaFX board/grid controls
- selected row/column

**Constructor:**

- CheckersView(game)

**Main methods you will likely need:**

- drawBoard()
- drawPieces()
- handleSquareClick(row, col)
- refresh()
- showTurn()
- showWinner()

**Matching test file:** CheckersViewTest.java (only test logic you can reasonably separate from JavaFX)

## GameRecorder.java

**What it is:** Records the moves made during a game so the game can be saved or replayed later.

**Fields / data it needs:**

- List of Move objects
- optional player/result information

**Constructor:**

- GameRecorder()

**Main methods you will likely need:**

- recordMove(move)
- getMoves()
- clear()
- saveGame(file)

**Matching test file:** GameRecorderTest.java

## GameReplay.java

**What it is:** Reads a saved game and plays the recorded moves back in order.

**Fields / data it needs:**

- List of recorded moves
- current replay position/index
- CheckersBoard or CheckersGame reference

**Constructor:**

- GameReplay(savedMoves) or GameReplay(file)

**Main methods you will likely need:**

- loadGame(file)
- nextMove()
- hasNextMove()
- resetReplay()

**Matching test file:** GameReplayTest.java

## CheckersApp.java

**What it is:** Starts the JavaFX program. This should stay small and mainly connect the game logic to the view.

**Fields / data it needs:**

- Usually only the main game/view references

**Constructor:**

- JavaFX creates this class for the application

**Main methods you will likely need:**

- main(String[] args)
- start(Stage stage)

**Matching test file:** CheckersAppTest.java is usually minimal because this class mainly launches JavaFX

## ComputerPlayer.java

**What it is:** Chooses a legal move for the computer opponent.

**Fields / data it needs:**

- computerColor
- reference to game/board if needed

**Constructor:**

- ComputerPlayer(color)

**Main methods you will likely need:**

- chooseMove(board)
- getLegalMoves(board)
- chooseCaptureIfAvailable(...)

**Matching test file:** ComputerPlayerTest.java

## Simple build order

Piece.java -> Move.java -> CheckersBoard.java -> MoveValidator.java -> CheckersGame.java -> CheckersView.java -> GameRecorder.java / GameReplay.java -> CheckersApp.java -> ComputerPlayer.java

**Important:** You do not have to create every method exactly as written here. This is a simple planning guide so you know what each file needs to accomplish. As you program, we can work through one class and one method at a time.